

Project Proposal

Title

GraphGuard: Adversarial Subgraph-Aware Oversampling for Minority-Class Fraud Detection in Transaction Graphs

Domain

Graph Machine Learning applied to **Financial Fraud Detection**, specifically fraud/illicit-activity detection on **cryptocurrency transaction networks**. Sits at the intersection of:

- Graph Neural Networks (GNNs)
 - Generative Adversarial Networks (GANs)
 - Class-imbalanced learning
-

Problem Statement

Fraudulent transactions in a network like Bitcoin are extremely rare compared to legitimate ones (often under 2–3% of all transactions). When we model these transactions as a graph — transactions as nodes, fund flows as edges — a GNN classifier trained on this data tends to just predict "legitimate" for everything, since that gets it ~98% accuracy while completely missing the fraud it's supposed to catch.

Existing fixes have gaps:

- **Reweighting the loss function** — a blunt fix, doesn't create new information, hard to tune correctly.
- **SMOTE-style oversampling (e.g. GraphSMOTE)** — generates new minority nodes by interpolating features, and predicts edges separately, but isn't adversarial, so synthetic nodes aren't always structurally realistic.
- **Cost-sensitive GNNs (e.g. GAT-COBO)** — reweight misclassification cost instead of generating anything new.

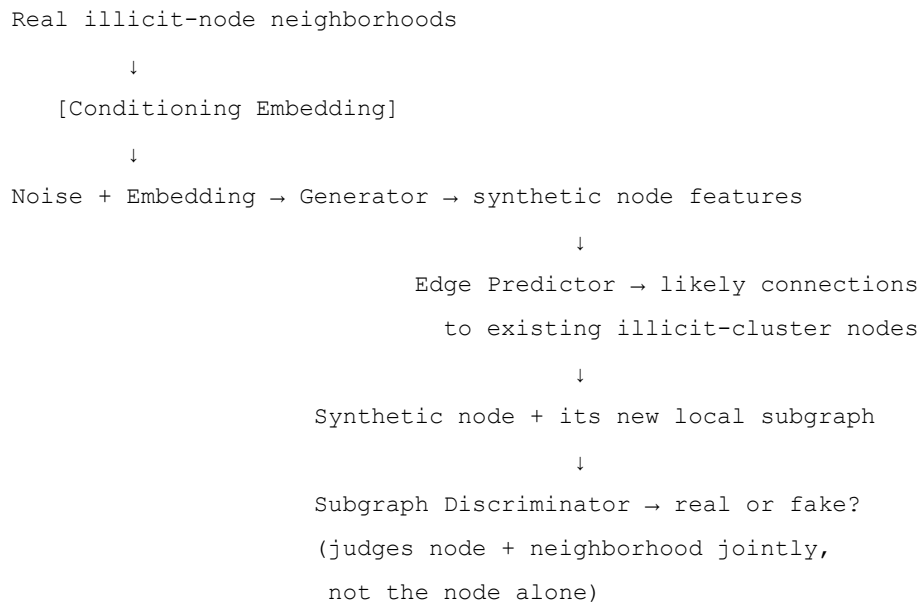
None of these **jointly and adversarially generate both the node's features AND its position in the fraud sub-network**, while judging realism at the **neighborhood level** rather than the isolated node level.

Our problem: Design a system that generates synthetic illicit-transaction nodes — complete with realistic features *and* realistic connections into existing fraud clusters — using a GAN whose discriminator evaluates the generated node together with its local subgraph, not in isolation. This should give a downstream GNN classifier a richer, more balanced training signal without introducing structurally implausible nodes.

Methodology & Architecture

Step 1 — Baseline Train a standard GraphSAGE classifier directly on the imbalanced graph. This establishes the "before" numbers (expect high accuracy, poor recall/F1 on the illicit class) — the problem we're fixing.

Step 2 — Graph-Conditioned GAN (the novel component)



- **Generator:** takes random noise + an embedding summarizing a real illicit node's neighborhood, outputs a synthetic feature vector.
- **Edge Predictor:** decides which existing nodes (mostly within illicit clusters) the synthetic node should connect to — preserving k-hop structural plausibility.
- **Discriminator:** instead of scoring just the synthetic node's features, it scores the **synthetic node + its freshly attached local subgraph** — forcing the generator to produce nodes that fit believably into fraud clusters, not just nodes with fraud-like features floating in isolation.
- **Optional enhancement:** add a contrastive loss term inside the discriminator to sharpen its ability to separate real vs. fake substructures (inspired by intra-class substructure differentiation approaches in recent literature).

Step 3 — Augmented Training Merge the real graph with the synthetic minority nodes/edges. Retrain the GraphSAGE (or GAT) classifier on this balanced graph.

Step 4 — Evaluation Compare against baselines (plain GCN, GraphSMOTE, cost-sensitive GAT-COBO) using:

- F1-score on the illicit class specifically
- Recall (catching actual fraud)
- Precision
- AUC-ROC

Datasets

Dataset

Purpose

Dataset	Purpose
Elliptic Bitcoin Dataset (Kaggle)	Primary dataset — ~203,769 transactions, 234,355 edges, 166 features/node, 49 time steps, labeled illicit/licit/unknown
Elliptic++ (extended actor-level graph, if available)	Secondary/generalization test — adds wallet-actor level graph on top of transaction-level graph
IEEE-CIS Fraud Detection (Kaggle)	Optional cross-domain validation — credit card fraud, tests if the method generalizes beyond crypto

Relevant Patents Reviewed

(Titles to be filled in later)

Patent Number	Patent Title
CN 115204901 B	
US 11631029 B2	
CN 120750580 A	
US 12119982 B2	
US 2025/0036956 A1	
CN 121936537 A	
CN 121936537 A	
US 12511659 B2	
CN 118761328 A	
CN 119378597 A	
US 10003605 B2	
US 11403643 B2	
US 2024/0062041 A1	

Plain-Language Summary

Imagine a huge web of Bitcoin transactions, where each transaction is a dot, and lines connect dots that sent money to each other. Somewhere in this web, criminals are hiding — moving stolen or illegal money through a chain of transactions to make it look clean. This is called money laundering.

The problem is: out of hundreds of thousands of transactions, only a tiny fraction — maybe 2 out of 100 — are actually fraudulent. If we train a computer model to spot fraud, it can "cheat" by just guessing "not fraud" every single time, and still be right 98% of the time. That sounds good on paper, but it means the model catches almost no real fraud — which defeats the whole purpose.

The usual fix is to show the model more examples of fraud by copying or slightly tweaking the few real fraud cases we have. But there's a catch: fraud isn't just about *what a transaction looks like* — it's also about *who it's connected to*. A fraudulent transaction usually sits inside a cluster of other suspicious transactions. If we create a fake "fraud example"

but don't also give it realistic connections to other suspicious transactions, the model doesn't learn much from it — it's like showing someone a photo of a criminal with no context about who they associate with.

Our idea is to build a system with two competing AI parts (this is what a GAN is — a Generative Adversarial Network):

1. One part **creates** fake fraud examples — both the transaction itself and its connections to other suspicious transactions.
2. The other part **checks** whether that fake example, along with its connections, looks believable — like a detective checking not just a suspect but their whole circle of associates.

These two parts compete and improve each other over time, until the fake fraud examples become very realistic — both in isolation and in context. We then mix these realistic fake examples into our training data, so the fraud-detection model gets to see many more convincing examples of what fraud (and its surrounding network) looks like — without ever using real people's private data. This should make the final fraud detector much better at actually catching fraud, instead of just being "accurate" by ignoring it.